

Name: Vakkalanka Sai Bhaskara Sujit

Roll number: 2520090085

Practical Week 8

Registering the user in spring boot application:

```
PS C:\WINDOWS\system32> cd "C:\Users\DELL\OneDrive\Documents\DBSEIDBD\Practical\student-records\student-records"
PS C:\Users\DELL\OneDrive\Documents\DBSEIDBD\Practical\student-records\student-records> Invoke-RestMethod -Uri "http://localhost:8080/auth/register" -Method POST -ContentType "application/json" -Body '{"username":"Sujit","password":"password123"}'

message
-----
registered successfully
```

User Login and JWT Token Generation:

```
PS C:\Users\DELL\OneDrive\Documents\DBSEIDBD\Practical\student-records\student-records> Invoke-RestMethod -Uri "http://localhost:8080/auth/register" -Method POST -ContentType "application/json" -Body '{"username":"Sujit","password":"password123"}'

token      :
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJTamVqaXQiLCJpc3MiOiJkaWdpdGFsLWxpYnJhcnkiLCJpYXQiOiE3MTgxMTI5MDAsImV4cCI6MTcxODExNjUwMH0.abc123exampletoken...
```

Saving the JWT Token:

```
PS C:\Users\DELL\OneDrive\Documents\DBSEIDBD\Practical\student-records\student-records> $token = "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJwcmI5YW5rYSISImldCI6MTc5MDc..."
PS C:\Users\DELL\OneDrive\Documents\DBSEIDBD\Practical\student-records\student-records> $headers = @{
>>   Authorization = "Bearer $token"
>> }
```

GET All Students Records:

```
PS C:\Users\DELL\OneDrive\Documents\DB\EQDBD\Practical\student-records\student-records
> Invoke-RestMethod -Uri "http://localhost:8080/students" -Method GET -Headers $header

id      : 6ab0f91bf620804aca5e8a07
name    : Sujit
age     : 18
course  : CSIT
email   : 2520090085@klh.edu.in
```

Student Record Updated Successfully:

```
PS C:\Users\DELL\OneDrive\Documents\DB\SEDBD\Practical\student-records\student-records> Invoke-RestMethod -Uri "http://localhost:8080/students/6ab0d6e94bd72bb2efd336e4" -Method PUT -Headers $headers -ContentType "application/json" -Body {"name":"Sujit","age":18,"course":"CSIT","email":"2520090085@klh.edu.in"}

id      : 6ab0d6e94bd72bb2efd336e4
name    : Sujit
age     : 18
course  : CSIT
email   : 2520090085@klh.edu.in
```

Test Validation: Student Input Validation and Error Handling

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\student-records\student-records> Invoke-RestMethod -Uri "http://localhost:8080/students" -Method POST -Headers $headers -ContentType "application/json" -Body '{"name":"","age":10,"course":"","email":"wrong-email"}'
Invoke-RestMethod : The remote server returned an error: (400) Bad Request.
At line:1 char:1
+ Invoke-RestMethod -Uri "http://localhost:8080/students" -Method POST ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
```

Student Records Stored in MongoDB

Delete the student record

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\student-records\student-records> Invoke-RestMethod -Uri "http://localhost:8080/students/6ab0d6e94bd72bb2efd336e4" -Method DELETE -Headers $headers
```

```
test> use student_records
switched to db student_records
student_records> show collections
students
users
student_records> db.students.find().pretty()
[
  {
    _id: ObjectId('6ab0d6e94bd72bb2efd336e4'),
    name: 'Sujit',
    age: 18,
    course: 'CSIT',
    email: '25200900085@klh.edu.in',
    _class: 'com.example.student_records.entity.Student'
  }
]

student_records> |
```

Output on PowerShell 1: [Student Record Deleted Successfully]

```
2026-09-30T20:41:01.852+05:30 INFO 14660 --- [student-records]
[nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
JWT VALID
JWT VALID
```

Student data deletion verification

```
PS C:\Users\DELL\OneDrive\Documents\S DBE&DBD\Practical\student-records\student-records> Invoke-RestMethod -Uri "http://localhost:8080/students" -Method GET -Headers $headers
```

MongoDB Records After Deletion

```
test> use student_records
switched to db student_records
student_records> db.students.find().pretty()

student_records> exit
```

Codes:

1. Main Application

- StudentRecordsApplication.java

```
package com.example.student_records;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class StudentRecordsApplication {
    public static void main(String[] args) {
        SpringApplication.run(StudentRecordsApplication.class, args);
    }
}
```

2. Entity

- Student.java

```
package com.example.student_records.entity;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
@Document(collection="students")
public class Student {
    @Id
    private String id;
    private String name;
    private Integer age;
    private String course;
    private String email;
    public String getId() {
        return id;
    }
    public void setId(String id) {
```

```

        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public Integer getAge() {
        return age;
    }
    public void setAge(Integer age) {
        this.age = age;
    }
    public String getCourse() {
        return course;
    }
    public void setCourse(String course) {
        this.course = course;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}

```

- User.java

```
package com.example.student_records.entity;
```

```
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.index.Indexed;
import org.springframework.data.mongodb.core.mapping.Document;
```

```
@Document(collection = "users")
public class User {
```

```
    @Id
    private String id;
```

```
    @Indexed(unique = true)
    private String username;
```

```
    private String password;
```

```
    public String getId() {
        return id;
    }

```

```
    public void setId(String id) {
        this.id = id;
    }
}

```

```

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}

```

3. DTO

- StudentRequest.java

```

package com.example.student_records.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;

public class StudentRequest {

    @NotBlank(message = "name is required")
    private String name;

    @NotNull(message = "age is required")
    @Min(value = 16, message = "age must be at least 16")
    private Integer age;

    @NotBlank(message = "course is required")
    private String course;

    @NotBlank(message = "email is required")
    @Email(message = "a valid email is required")
    private String email;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

```
}

public Integer getAge() {
    return age;
}

public void setAge(Integer age) {
    this.age = age;
}

public String getCourse() {
    return course;
}

public void setCourse(String course) {
    this.course = course;
}

public String getEmail() {
```

```

        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}

```

- RegisterRequest.java

```

package com.example.student_records.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class RegisterRequest {

    @NotBlank
    private String username;

    @NotBlank
    @Size(min = 6, message = "password must be at least 6 characters")
    private String password;

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}

```

- LoginRequest.java

```
package com.example.student_records.dto;
import jakarta.validation.constraints.NotBlank;

public class LoginRequest {

    @NotBlank
    private String username;

    @NotBlank
    private String password;

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

4. Repository

- StudentRepository.java

```
package com.example.student_records.repository;

import com.example.student_records.entity.Student;
import org.springframework.data.mongodb.repository.MongoRepository;

public interface StudentRepository extends MongoRepository<Student, String>
{
}
```

- UserRepository.java

```
package com.example.student_records.repository;

import com.example.student_records.entity.User;
import org.springframework.data.mongodb.repository.MongoRepository;

import java.util.Optional;
```

```
public interface UserRepository extends MongoRepository<User, String> {

    Optional<User> findByUsername(String username);
}
```

5. Service

- StudentService.java

```
package com.example.student_records.service;

import com.example.student_records.dto.StudentRequest;
import com.example.student_records.entity.Student;
import com.example.student_records.repository.StudentRepository;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class StudentService {

    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public Student create(StudentRequest r) {

        Student s = new Student();

        apply(s, r);

        return repository.save(s);
    }

    public List<Student> getAll() {
        return repository.findAll();
    }

    public Optional<Student> getById(String id) {
        return repository.findById(id);
    }

    public Optional<Student> update(String id, StudentRequest r) {
```

```

        return repository.findById(id).map(s -> {

            apply(s, r);

            return repository.save(s);
        });
    }

    public boolean delete(String id) {

        if (!repository.existsById(id)) {
            return false;
        }

        repository.deleteById(id);

        return true;
    }

    private void apply(Student s, StudentRequest r) {

        s.setName(r.getName());
        s.setAge(r.getAge());
        s.setCourse(r.getCourse());
        s.setEmail(r.getEmail());
    }
}

```

6. Controller

- StudentController.java

```

package com.example.student_records.controller;

import com.example.student_records.dto.StudentRequest;
import com.example.student_records.entity.Student;
import com.example.student_records.service.StudentService;

import jakarta.validation.Valid;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/students")

```

```

public class StudentController {

    private final StudentService service;

    public StudentController(StudentService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<Student> create(
        @Valid @RequestBody StudentRequest r) {

        return ResponseEntity
            .status(201)
            .body(service.create(r));
    }

    @GetMapping
    public List<Student> getAll() {
        return service.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getById(
        @PathVariable String id) {

        return service.getById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PutMapping("/{id}")
    public ResponseEntity<Student> update(
        @PathVariable String id,
        @Valid @RequestBody StudentRequest r) {

        return service.update(id, r)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(
        @PathVariable String id) {

        return service.delete(id)
            ? ResponseEntity.noContent().build()
            : ResponseEntity.notFound().build();
    }
}

```

```
    }  
}
```

- AuthController.java

```
package com.example.student_records.controller;  
  
import com.example.student_records.dto.LoginRequest;  
import com.example.student_records.dto.RegisterRequest;  
import com.example.student_records.entity.User;  
import com.example.student_records.repository.UserRepository;  
import com.example.student_records.security.JwtUtil;  
  
import jakarta.validation.Valid;  
  
import org.springframework.http.ResponseEntity;  
  
import org.springframework.security.authentication.AuthenticationManager;  
import  
org.springframework.security.authentication.UsernamePasswordAuthenticationToken;  
  
import org.springframework.security.crypto.password.PasswordEncoder;  
  
import org.springframework.web.bind.annotation.*;  
  
import java.util.Map;  
  
@RestController  
@RequestMapping("/auth")  
public class AuthController {  
  
    private final UserRepository users;  
    private final PasswordEncoder encoder;  
    private final AuthenticationManager manager;  
    private final JwtUtil jwt;  
  
    public AuthController(  
        UserRepository users,  
        PasswordEncoder encoder,  
        AuthenticationManager manager,  
        JwtUtil jwt) {  
  
        this.users = users;  
        this.encoder = encoder;  
        this.manager = manager;  
        this.jwt = jwt;  
    }  
}
```

```

}

@PostMapping("/register")
public ResponseEntity<?> register(
    @Valid @RequestBody RegisterRequest r) {

    if (users.findByUsername(r.getUsername()).isPresent()) {

        return ResponseEntity
            .badRequest()
            .body(Map.of(
                "error",
                "username already taken"
            ));
    }

    User user = new User();

    user.setUsername(r.getUsername());

    user.setPassword(
        encoder.encode(r.getPassword())
    );

    users.save(user);

    return ResponseEntity
        .status(201)
        .body(Map.of(
            "message",
            "registered successfully"
        ));
}

@PostMapping("/login")
public ResponseEntity<?> login(
    @Valid @RequestBody LoginRequest r) {

    manager.authenticate(
        new UsernamePasswordAuthenticationToken(
            r.getUsername(),
            r.getPassword()
        )
    );

    return ResponseEntity.ok(
        Map.of(
            "token",

```

```

        jwt.generateToken(r.getUsername())
    )
    );
}
}

```

7. Security

- SecurityConfig.java

```

package com.example.student_records.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration
.AuthenticationConfiguration;

import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;

import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticat
ionFilter;

@Configuration
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;

    public SecurityConfig(JwtAuthFilter jwtAuthFilter) {
        this.jwtAuthFilter = jwtAuthFilter;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean

```

```

    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration configuration)
        throws Exception {

        return configuration.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain filterChain(
        HttpSecurity http)
        throws Exception {

        return http
            .csrf(csrf -> csrf.disable())

            .sessionManagement(session ->
                session.sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS
                )
            )

            .authorizeHttpRequests(auth ->
                auth
                    .requestMatchers("/auth/**")
                    .permitAll()

                    .anyRequest()
                    .authenticated()
                )

            .addFilterBefore(
                jwtAuthFilter,
                UsernamePasswordAuthenticationFilter.class
            )

            .build();
    }
}

```

- JwtUtil.java

```

package com.example.student_records.security;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;

import org.springframework.beans.factory.annotation.Value;

```

```

import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Date;

@Component
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration-ms}")
    private long expirationMs;

    private SecretKey key() {

        return Keys.hmacShaKeyFor(
            secret.getBytes(StandardCharsets.UTF_8)
        );
    }

    public String generateToken(String username) {

        Date now = new Date();

        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(now)
            .setExpiration(
                new Date(now.getTime() + expirationMs)
            )
            .signWith(key(), SignatureAlgorithm.HS256)
            .compact();
    }

    public String extractUsername(String token) {

        return Jwts.parserBuilder()
            .setSigningKey(key())
            .build()
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
    }

    public boolean isValid(String token) {

```

```

        try {

            Jwts.parserBuilder()
                .setSigningKey(key())
                .build()
                .parseClaimsJws(token);

            return true;

        } catch (JwtException | IllegalArgumentException e) {

            return false;
        }
    }
}

```

- JwtAuthFilter.java

```

package com.example.student_records.security;

import jakarta.servlet.*;
import jakarta.servlet.http.*;

import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtUtil jwtUtil;
    private final UserDetailsService userDetailsService;

    public JwtAuthFilter(
        JwtUtil jwtUtil,
        UserDetailsService userDetailsService) {

        this.jwtUtil = jwtUtil;
    }
}

```

```

        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain chain)
        throws ServletException, IOException {

        String header = request.getHeader("Authorization");

        if (header != null && header.startsWith("Bearer ")) {

            String token = header.substring(7);

            if (jwtUtil.isValid(token)) {

                System.out.println("JWT VALID");

                String username =
                    jwtUtil.extractUsername(token);

                var details =
                    userDetailsService.loadUserByUsername(username);

                var authentication =
                    new UsernamePasswordAuthenticationToken(
                        details,
                        null,
                        details.getAuthorities()
                    );

                authentication.setDetails(
                    new WebAuthenticationDetailsSource()
                        .buildDetails(request)
                );

                SecurityContextHolder
                    .getContext()
                    .setAuthentication(authentication);

            } else {

                System.out.println("JWT INVALID");
            }
        }
    }

```

```

        chain.doFilter(request, response);
    }
}

```

- CustomUserDetailsService.java

```

package com.example.student_records.security;

import com.example.student_records.repository.UserRepository;

import org.springframework.security.core.userdetails.*;
import org.springframework.stereotype.Service;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {

        var user = userRepository
            .findByUsername(username)
            .orElseThrow(() ->
                new UsernameNotFoundException("User not found"));

        return User.withUsername(user.getUsername())
            .password(user.getPassword())
            .authorities("USER")
            .build();
    }
}

```

8. Exception Handling

- GlobalExceptionHandler.java

```

package com.example.student_records.exception;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;

```

```

import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation(
        MethodArgumentNotValidException ex) {

        Map<String, String> errors = new HashMap<>();

        ex.getBindingResult()
            .getFieldErrors()
            .forEach(error ->
                errors.put(
                    error.getField(),
                    error.getDefaultMessage()
                )
            );

        Map<String, Object> body = new HashMap<>();

        body.put("error", "VALIDATION_ERROR");
        body.put("details", errors);

        return ResponseEntity.badRequest().body(body);
    }
}

```

9. Configuration

```

application.properties

spring.application.name=student-records

spring.data.mongodb.uri=mongodb://localhost:27017/student_records

jwt.secret=replace-this-with-a-long-random-secret-at-least-32-characters

jwt.expiration-ms=3600000

server.port=8080

```

10. Maven Configuration

- pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.6</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>student-records</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>student-records</name>
    <description>Student Records Application</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-mongodb</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <dependency>
```

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.11.5</version>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.11.5</version>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.11.5</version>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>

</project>

```